

1 Implémentation d'un mécanisme de protection-reprise

Dans ce TP, nous allons implémenter un mécanisme de protection-reprise qui permettra de reprendre un calcul là où le code s'était arrêté. Comme le code tourne en parallèle, nous souhaitons que les écritures et lectures soient faites en parallèles. Pour se faire, nous allons utiliser les fonctionnalités des sous-programmes MPI-IO.

Le code est celui du *jeu de la vie* modélisant l'évolution d'un ensemble de cellules dont leur état suit différentes règles pré-établies. L'état d'une cellule au temps suivant dépend de l'état actuel de ces voisins. Au cours de la simulation, une cellule peut être active (vivante) ou inactive (morte).

Le code source est présent dans le dossier `life-io/` :

- Sur le calculateur, chargez un module MPI¹ :

```
> module avail mpi,  
> module load mpi/...
```
- Pour compiler le code, un `Makefile` est disponible dans le dossier du code source. Il existe deux versions du code : le premier, appelé `mlife`, est fonctionnel dès le début du TP et affiche le résultat de la simulation en sortie-écran. Le second, appelé `mlife-mpiio`, est la version du code faisant appel à MPI-IO et sera à compléter tout au long du TP.

```
> make mlife  
> make mlife-mpiio
```
- De manière générale, le code se lance de la manière suivante :

```
> srun -n <nb_processes> mlife/mlife-mpiio -x <dim_x> -y <dim_y> -i <nb_iter>  
-r <restart_iter> -p <checkpoint_filename>
```
- Pour la version `mlife` :

```
> srun -n 4 mlife -x 50 -y 50 -i 20 -r -1 -p useless
```
- Pour la version `mlife-mpiio`, lorsque la protection est activée :

```
> srun -n 4 mlife-mpiio -x 50 -y 50 -i 20 -r -1 -p my_io_save
```
- Pour la version `mlife-mpiio`, lorsque la reprise est activée (ici, le calcul reprend à l'itération 19):

```
> srun -n 4 mlife-mpiio -x 50 -y 50 -i 30 -r 19 -p my_io_save
```

Question1

Au sein de la simulation, l'ensemble des cellules est représenté sous forme d'une grille en deux dimensions. Afin de savoir précisément ce qui est nécessaire d'être sauvegarder dans le fichier de protection, nous allons tout d'abord analyser comment fonctionne le code.

- Comment est définie cette grille localement pour chaque processus MPI ? A quoi ressemble cette grille sur l'ensemble des processus MPI ?
- Comment est gérée la frontière de la grille pour chaque processus MPI ? Que contient cette frontière à l'initialisation puis dans la boucle des itérations ?
- A-t-on besoin de communications au sein de la simulation entre les processus MPI. Si oui, pourquoi ?

¹La version `openmpi` permet d'avoir accès à un manuel en ligne permettant de connaître l'ensemble des paramètres d'un sous-programme MPI : `man MPI_<nom_programme>`.

Question2

Dans cette partie, nous allons implémenter l'écriture parallèle du fichier de protection. Compléter les zones dans l'ordre, et vérifier le fonctionnement du mécanisme.

- (a) Duplication et libération du communicateur :
 - Zone-1-A communicator duplication : comm into mlifeio_comm (MPI_Comm_dup).
 - Zone-1-A-bis communicator duplication : free mlifeio_comm communicator (MPI_Comm_free)
- (b) Ouverture et fermeture du fichier de protection :
 - Zone-1-B define the correct opening mode for writing the protection file.
 - Zone 1-B-bis open a MPI_file fh named "filename", with amode (MPI_File_open).
 - Zone 1-B-ter close a MPI_file fh (MPI_File_close)
- (c) Typage des données écrites dans le fichier de protection :
 - Zone-1-C Create in vectype a MPI_Type_vector for the desired portion of the matrix
 - Zone-1-C-bis Position vectype so that it covers the desired data in the matrix (MPI_Get_address, MPI_Type_create_hindexed)
 - Zone-1-C-ter Free vectype (MPI_Type_free)
- (d) Création d'une structure MPI pour le processus 0 qui contiendra également les méta-données :
 - Zone-1-D Create in newtype a MPI structure (MPI_Type_create_struct, MPI_Get_address, MPI_BOTTOM)
 - Zone-1-D-bis free rowblk
- (e) Ecriture du fichier de protection :
 - Zone-1-E compute the file offset for process 0
 - Zone-1-E-bis compute the file offset for non-0 processes
 - Zone-1-E-ter commit the new type so that it can be used (MPI_Type_commit)
 - Zone-1-E-quater Explicitly write the data at the computed offset in file, use MPI_File_write_at_all
 - Zone-1-E-quinquies free type)

Question3

Maintenant, nous allons traiter la lecture du fichier (reprise). Compléter les zones dans l'ordre, et vérifier le fonctionnement du mécanisme.

- (a) Ouverture et fermeture du fichier de protection à lire :
 - Zone-2-A define the correct opening mode for reading the protection file
 - Zone 2-A-bis open a MPI_file fh named "filename", with amode
 - Zone 2-A-ter close a MPI_file fh
- (b) Lecture de l'en-tête du fichier et vérification :
 - Zone 2-B make all processes read the header of the file : it contains 3 integers (rows, columns, iterations), use MPI_File_read_at_all
 - Zone 2-B-bis use an allreduce max on err to check if there was an error in the read.
- (c) Lecture de la grille :
 - Zone 2-C compute the offset in the file (myfileoffset) to access the desired data (use myoffset)
 - Zone 2-C-bis commit the type and read the selected data (type and offset), use MPI_Type_commit and MPI_File_read_at_all
 - Zone 2-C-ter free type